

# Clean up the event records

Student experiment booklet | Bash in GitHub Codespaces | About 100 minutes

Name: \_\_\_\_\_ Lab ID: \_\_\_\_\_ Section: \_\_\_\_\_

Your event team has draft notices, repeated registrations and conflicting timetables. Learn seven tools, investigate the records, and prepare a reliable set of files.

## Get ready

Sign in with your GitHub account. Your instructor must grant access while this repository is private. Open the Lab 2 link below and create a Codespace on **main**, or resume your Lab 2 Codespace. Lab 1 uses a different repository.

[Open the Lab 2 Codespace](https://github.com/nalinabrol/terminal-lab-2) | [github.com/nalinabrol/terminal-lab-2](https://github.com/nalinabrol/terminal-lab-2)

Wait for setup. Choose **Trust Folder & Continue** if asked. A maximized Bash terminal opens after startup. At the repository root, run:

```
bash terminal-lab-2/start.sh
```

Enter your instructor-assigned ID. Run the exact **cd** command printed by setup, then run **pwd** and **cat identity.txt**. Reusing an ID preserves the existing attempt; it does not reset mistakes.

My lab root (the full path printed by pwd):

EVENT: \_\_\_\_\_ ROOM: \_\_\_\_\_ TIME: \_\_\_\_\_

| Folder     | Purpose                                                         |
|------------|-----------------------------------------------------------------|
| practice/  | Original evidence for guided experiments. Keep it unchanged.    |
| challenge/ | Fresh evidence for your independent mission. Keep it unchanged. |
| work/      | Your working copies and saved results. Already created for you. |

**How to work:** predict, try, inspect, and explain. Begin every experiment at your lab root. Type one command at a time. Use the terminal and nano for edits; do not generate solution scripts. Write observations, not entire screen transcripts.

Hints: page 12. Command reference: page 13. Save your work in this Codespace. The checker does not submit files. No permissions, environment-variable, or scripting lesson is required.

# Find a command you used before

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

## 1A | Make a small trail

Type and run these commands **one at a time**. They provide a known trail to search in your own terminal history.

```
grep -F "EVENT=" identity.txt
grep -F "ROOM=" identity.txt
history 10
```

**history** lists remembered commands with numbers. **history 10** shows up to ten recent entries. Your numbers may differ from a classmate's. Lab 1 used Up Arrow; today you will search instead of stepping through every command.

Which number is beside your ROOM search? \_\_\_\_\_

## 1B | Filter the history list

```
history | grep -F "identity.txt"
```

This reuses Lab 1's pipe and text search to search **command text**. The history-search command itself may also appear because it contains identity.txt.

Write one earlier matching command, excluding the history search itself:

## 1C | Search interactively, then edit

At an empty Bash prompt, press **Ctrl+R**. Type **grep -F** slowly. The matching command appears. Press **Ctrl+R** again to find an older match. Stop at the command that searches ROOM or EVENT.

Press **Right Arrow** to accept that command for editing without running it. Use the arrow keys and Backspace to change the quoted search text to **TIME=**. Inspect the whole line, then press Enter.

What did your edited command print?

## 1D | Leave a search without running it

From an empty prompt, press **Ctrl+R**, type **identity**, then press **Ctrl+G**. This cancels the search and restores the input you had before it. No matched command should run.

Click inside the terminal before using shortcuts. Use **Control**, not Command, on a Mac. If the browser intercepts a shortcut, use the history-list method and ask your instructor for help.

Show your instructor: find an older command, edit it, and explain before running it.

# Repair a notice with nano

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

**nano** is a text editor inside the terminal. Unlike echo with redirection, it lets you move to and edit an existing line.

## 2A | Inspect before changing

```
cat identity.txt
cat work/notice.txt
nano work/notice.txt
```

Use the arrow keys to move. Change **Room: Hall-0** to your ROOM from identity.txt. Keep the text **Room:** before the value. Do not change the event code.

| Keys                   | In nano                                                                 |
|------------------------|-------------------------------------------------------------------------|
| Arrow keys / Backspace | Move the cursor / remove the character before it.                       |
| Ctrl+O, then Enter     | Write the file; confirm its existing filename.                          |
| Ctrl+X                 | Exit nano. If asked to save changes, choose Y and confirm the filename. |

Nano displays Control shortcuts with a caret: **^O** means Ctrl+O. It does not mean type the two characters ^ and O.

## 2B | Save, exit, and verify

Save your room change and exit. Run **cat work/notice.txt**. Then reopen it in nano to confirm that the saved change is still there.

What evidence shows that your edit was saved?

## 2C | Finish independently

Update **Doors open:** to your TIME from identity.txt. Add a fourth line containing exactly **Bring your student card**. Save and exit. Your file must contain exactly four lines in the order shown below, using your own values:

```
Event: YOUR_EVENT
Room: YOUR_ROOM
Doors open: YOUR_TIME
Bring your student card.
```

YOUR\_EVENT, YOUR\_ROOM and YOUR\_TIME above are placeholders, not text to leave in the file. Finish the final line with Enter so it ends with a newline.

My final room: \_\_\_\_\_ My final opening time: \_\_\_\_\_

# Find the current instruction

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

A long file is hard to inspect with `cat`. **less** lets you scroll and search without changing the file.

```
less practice/handbook.txt
```

| Keys                    | In less                                                   |
|-------------------------|-----------------------------------------------------------|
| Space / b               | Move forward / backward by one screen.                    |
| g / G                   | Jump to the beginning / end. Use Shift+g for uppercase G. |
| /COLLECTION, then Enter | Search forward for COLLECTION.                            |
| n / N                   | Next / previous match of that search.                     |
| q                       | Quit less and return to Bash.                             |

## 3A | Predict, then compare

Start at the beginning with **g**. Search for **COLLECTION**. Read the heading and the next lines. Is the first matching instruction necessarily the correct one?

First heading found:

Use **n** to find later matches. Use **N** to go back once. Find the heading **COLLECTION - ACTIVE** with your **EVENT** code.

Why should the archived and other-event instructions be rejected?

## 3B | Save the answer

Write down the complete sentence immediately below the **ACTIVE** heading. Quit less with **q**. Open **nano work/collection.txt** and save only that sentence on one line. Keep its capitalization and final full stop; finish with **Enter**.

The approved collection sentence:

## 3C | Explain the choice of tool

How does **less** help with this task compared with displaying the whole file using **cat**?

If you see a colon or (END), you may still be inside **less**. Press **q** before entering a shell command.

# Put records in order

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

## 4A | Sort names

```
cat practice/names.txt
```

Before sort: predict the first and last names. First: \_\_\_\_\_ Last: \_\_\_\_\_

```
sort practice/names.txt
```

**sort** writes ordered lines to the terminal. It does not rewrite its input file. These files use simple names with consistent capitalization.

Save the alphabetically sorted names to **work/names-sorted.txt**, then inspect the saved file. Use Lab 1's output redirection.

My command:

## 4B | Are numbers just text?

```
cat practice/scores.txt
```

Predict both orders before running the two sort commands below. Ordinary sort compares text; **-n** compares numeric values. For this file, each line contains just one non-negative whole number.

Ordinary sort order:

Numeric sort order:

```
sort practice/scores.txt
sort -n practice/scores.txt
```

## 4C | Highest first

**-r** reverses the order. Combine it with **-n** to sort these scores from highest to lowest. Save the result as **work/scores-desc.txt**.

My command:

Largest score: \_\_\_\_\_ Smallest score: \_\_\_\_\_

Inspect practice/scores.txt again. Its original order should remain unchanged. Never use the same file as both the input and the destination of **>**.

# When does a duplicate disappear?

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

```
cat practice/repeated-names.txt
```

Some repeated names are next to each other; others are separated. **uniq** compares adjacent lines.

## 5A | Predict the result

Which name will still appear more than once after plain **uniq**?

```
uniq practice/repeated-names.txt
```

Observed repeated name(s):

## 5B | Bring matching lines together

```
sort practice/repeated-names.txt | uniq
```

Sorting brings identical names together. Now **uniq** can keep one line from each group. Save this output to **work/unique-names.txt**.

Why did sorting first change the result?

## 5C | Count each group

```
sort practice/repeated-names.txt | uniq -c
```

**-c** adds the number of lines in each adjacent group. Leading spaces before a count are normal. Save the counts to **work/name-counts.txt**.

One name and its count: \_\_\_\_\_ Count: \_\_\_\_\_

Add the counts mentally and compare the sum with the number of lines in the original file. This counts entries, not necessarily different people: the names in this small dataset identify fictional participants.

Sum of counts: \_\_\_\_\_ Original line count: \_\_\_\_\_

**Independent check:** explain why using **uniq** before **sort** is not enough to guarantee one line per name.

# Choose the fields you need

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

Each record in this file has three fields: **student ID | name | workshop**. There is no header row. The vertical bar separates fields; it is the delimiter.

```
head -n 3 practice/registrations.txt
cut -d "|" -f 2 practice/registrations.txt
```

**cut -d "|"** selects the field separator; **-f 2** selects field 2. Quote the separator so Bash passes it to cut instead of treating it as a pipeline.

From the first record: field 1 = \_\_\_\_\_ field 2 = \_\_\_\_\_ field 3 = \_\_\_\_\_

## 6A | Save the names

Extract every name to **work/registration-names.txt**. Preserve the original row order and repetitions. Do not sort this output.

My command:

## 6B | Select more than one field

```
cut -d "|" -f 1,3 practice/registrations.txt
```

Save that result to **work/id-workshop.txt**. Notice that cut keeps the delimiter between the selected fields.

What is missing from this result compared with the original file?

## 6C | Count workshop registrations

Extract workshop choices into **work/workshops.txt**. Then sort that file and use **uniq -c**. Save the counts to **work/workshop-counts.txt**. **work/workshops.txt** is an intermediate file, not a separate graded result.

My extraction command:

My sorting-and-counting command:

How does cut differ from grep?

These are simple delimiter-separated records. We are not handling quoted CSV fields, missing fields, or delimiters inside names in this exercise.

# What changed between versions?

**Start:** your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

## 7A | Compare an identical pair

```
diff practice/timetable-before.txt work/timetable.txt
```

**diff FIRST SECOND** reports line differences. No output means no differences were found. A missing file produces an error instead.

What did you observe?

-----

## 7B | Compare the old and approved versions

```
diff practice/timetable-before.txt practice/timetable-approved.txt
```

| Marker    | Meaning in this output                                  |
|-----------|---------------------------------------------------------|
| <         | This line is from the FIRST file (old timetable).       |
| >         | This line is from the SECOND file (approved timetable). |
| a / c / d | In the numbered instruction: add / change / delete.     |
| ---       | Separates the old and new lines in a change block.      |

For example, **2c2** means line 2 in the first file changes to line 2 in the second. **3d2** means remove line 3 from the first file; the second-file position is after line 2. Focus on the event lines under the markers.

Which event moved to a different time?

-----

Which event was removed? Which was added?

-----

## 7C | Repair the working copy

Open **nano work/timetable.txt**. Make the three required changes so it matches the approved timetable. Save and exit, then compare:

```
diff work/timetable.txt practice/timetable-approved.txt
```

What result tells you the repair is complete?

-----

You can save a comparison with **>**, just as in Lab 1. Do not add its output to either input file. A comparison with differences is expected here; it is not a broken command.

# Prepare the event for opening

Use only the fresh evidence in **challenge/** and working copies in **work/challenge/**. These files were created at setup, so this mission does not depend on finishing the guided experiments. Begin at your lab root.

## Your mission

- 1. Discover.** Open **challenge/brief.txt** in less. Find **APPROVAL - CURRENT** for your EVENT. Ignore the old approval. It specifies four exact announcement lines.
- 2. Repair.** Use nano to correct **work/challenge/announcement.txt** to those four lines. Preserve capitalization, punctuation and line order.
- 3. Extract and organise.** Records in **challenge/registrations.txt** use the same three-field format as Experiment 6 and have no header. Produce an alphabetically sorted list of distinct names as **work/challenge/participants.txt**.
- 4. Count.** Count the number of registration rows for each workshop. Save a uniq -c style report as **work/challenge/workshop-counts.txt**. Count all registrations, including repeated names.
- 5. Compare.** Use diff with **challenge/schedule-before.txt** FIRST and **challenge/schedule-approved.txt** SECOND. Save its normal comparison output as **work/challenge/schedule-diff.txt**.
- 6. Repair and verify.** Use nano to update **work/challenge/timetable.txt** to match the approved schedule. Compare them again to check your edits.

## Required results

```
work/challenge/  
  announcement.txt  
  participants.txt  
  workshop-counts.txt  
  schedule-diff.txt  
  timetable.txt
```

Use intermediate files in **work/challenge/** if they help. Several clear commands are as valid as one long pipeline. Preserve every original file in **practice/** and **challenge/**.

Before beginning: which tools will you combine for the participant list?

---

---

# Show why your results are right

## Inspect your five files

| Result             | My evidence                                                     |
|--------------------|-----------------------------------------------------------------|
| Announcement       | Correct event, room, opening time and final instruction.        |
| Participants       | Alphabetical order; no name occurs twice.                       |
| Workshop counts    | Counts include every registration row.                          |
| Saved comparison   | Old schedule was the first input; approved schedule was second. |
| Repaired timetable | Comparison with the approved file prints no differences.        |

My approved event and room:

Number of distinct names: \_\_\_\_\_ Total registrations: \_\_\_\_\_

## Explain your decisions

1. Why did sorting come before removing repeated names?

---

---

2. Which field did you select for workshop counts? What does each count represent?

---

---

3. Which session changed time, which was removed, and which was added?

---

---

## Use history again

Find a command you used during this challenge using history search. Bring it back for inspection without running it. Show your instructor what it reads and where it writes. Cancel or clear it afterwards.

The command I chose and why:

---

---

# Check, correct, and explain

From your lab root, run the checker below. A CHECK line tells you to inspect that result. It does not repair your work.

```
python3 ../../terminal-lab-2/check.py
```

The checker reads 15 result files and checks that the original evidence is unchanged: **16 checks in total**. It does not upload, submit, or change files. The checker is visible; a passing score alone does not prove how you worked.

My score: \_\_\_\_\_ / 16 | One issue I found and corrected:

.....

.....

## Short understanding check

1. Why can plain sort place 100 before 2? Which option fixes numerical order?

.....

.....

2. A name appears on lines 1 and 5. Will plain uniq necessarily remove one? Why?

.....

.....

3. diff prints nothing. What does that establish about the two files?

.....

.....

## Show a skill

Your instructor may ask you to do one of these on a fresh file or with a different search: find and edit an earlier command; save a correction in nano; or locate a later matching section in less.

Skill demonstrated: \_\_\_\_\_ Instructor check: \_\_\_\_\_

## Before leaving

- ☐ I inspected my saved files. ☐ I completed the written explanations.
- ☐ I preserved the original evidence. ☐ I handed in the booklet as instructed.
- ☐ I kept my Codespace and lab-work folder for review.

Do not delete or reset your Codespace. Follow your instructor's submission instructions. The checker does not submit on your behalf.

# Use one hint, then try again

## History

Start with **history 10**. To narrow the list, pipe history into grep. For Ctrl+R, click the terminal first, use Control (not Command), and type a fragment of a command you actually ran. Right Arrow accepts a match for editing; Enter runs it; Ctrl+G cancels the search.

## nano

The file path appears at the top of nano. Use arrows to reach the wrong text. Ctrl+O means write/save, followed by Enter to confirm the filename. Ctrl+X exits. Reopen the file or use cat to verify the result.

## less

Press g before searching to begin at the start. Type / followed by the search word and Enter. Use n for a later match. Read the surrounding heading: archived, active and other-event entries are different. Press q before typing a shell command.

## sort

Plain sort orders text. Add -n for numerical order and -r to reverse it. To save the output, put > and the destination after the command. Input and destination must be different files.

## uniq

If repetitions remain, inspect whether identical lines were next to one another. Put sort before uniq. Add -c to uniq for counts. Padding spaces before numbers are normal.

## cut

Count fields from 1. The name is field 2 and workshop is field 3. -d chooses the delimiter and -f chooses fields. Quote "|" so Bash does not interpret that character as a pipe. Selecting fields preserves row order.

## diff

Lines with < come from the first file; lines with > come from the second. Repair a working copy, not the original evidence. Compare the working copy with the approved version after saving.

## Challenge

Separate the tasks: find instructions, edit the announcement, extract names, remove repetition, count workshop choices, compare schedules, and repair the timetable. Use intermediate files if a long pipeline is hard to reason about.

A CHECK result can mean a missing file, wrong contents, or a changed original. Inspect the named file and the exact requirement. Ask for help before attempting to restore original evidence.

# Keep beside your terminal

Examples are generic. Use the paths and values required by each exercise. Do not type the \$ prompt or explanatory labels.

| Command or keys                                              | Purpose                                                        |
|--------------------------------------------------------------|----------------------------------------------------------------|
| history<br>history 10                                        | List remembered commands / show up to ten recent commands.     |
| history   grep -F "fragment"                                 | Search command text. The search command itself may appear.     |
| Ctrl+R, then text<br>Ctrl+R again                            | Search history backwards / find an older match.                |
| Right Arrow / Enter / Ctrl+G                                 | During history search: accept for editing / run / cancel.      |
| nano work/note.txt                                           | Open an existing file or create a new text file.               |
| Ctrl+O, Enter / Ctrl+X                                       | In nano: save and confirm filename / exit.                     |
| less practice/handbook.txt                                   | Read a file interactively without editing it.                  |
| Space / b / g / G                                            | In less: forward / backward / beginning / end.                 |
| /word, Enter / n / N / q                                     | In less: search / next / previous / quit.                      |
| sort names.txt<br>sort -nr scores.txt                        | Alphabetical text order / descending numerical order.          |
| sort names.txt   uniq<br>sort names.txt   uniq -c            | One line per distinct name / count each name.                  |
| cut -d " " -f 2 records.txt<br>cut -d " " -f 1,3 records.txt | Extract field 2 / fields 1 and 3 with a   separator.           |
| diff before.txt after.txt                                    | Compare files; < belongs to before and > belongs to after.     |
| diff before.txt after.txt > changes.txt                      | Save a normal comparison to a separate file.                   |
| command > work/result.txt<br>first   second                  | Lab 1: save output (replace destination) / pass output onward. |

Ctrl means Control on both Windows/Linux and Mac. Click the terminal before shortcuts. Commands run in Bash; nano and less have their own keys while open.